

MASTER TECHNICAL SPECIFICATION

Multi-Operator Social Commerce Platform
Version 2.0 — Refined Specification

Status: Authoritative Specification
Date: March 2026

This document supersedes Master Technical Specification v1.2
Confidential

Table of Contents

1. Platform Overview

The platform is a multi-tenant social commerce and distribution system enabling Network Operators to run storefronts, Vendors to supply product catalogs, Creators to promote products, and Customers to purchase products.

Each storefront operates under a separate domain and operator context, while users maintain one global account across the platform.

The platform combines capabilities of multi-vendor marketplaces, affiliate platforms, social commerce networks, and distribution systems.

The platform must support: multi-tenant storefronts, affiliate promotion, commission splitting, coupon funding by different parties, subscription-based platform monetization, multi-party payments, enterprise-grade compliance and auditing, and internationalized commerce.

2. Core Principles

1. Admin never sells or owns a storefront.
2. Only Network Operators own storefronts.
3. Vendors own product catalogs.
4. Creators promote products but do not fulfill orders.
5. Customers purchase products within storefront domains.
6. One user account can have multiple roles.
7. Orders are always scoped to storefront domains.
8. Money flows are ledger-based and append-only.
9. No party may incur a financial loss due to coupons or commissions.
10. Stripe webhooks determine payment finality.
11. Multi-tenant isolation must be enforced in all queries.
12. All public APIs must be versioned and maintain backward compatibility for at least two major versions.
13. Data sovereignty and residency constraints must be respected per operator jurisdiction.
14. All mutating financial operations must be idempotent.

NEW: Principles 12–14 added to address API stability, data residency, and financial idempotency gaps.

3. User Roles

3.1 Admin

Admin manages the platform and has access to all data. Responsibilities include platform configuration, operator approval, subscription plan management, dispute resolution, and compliance and audit review. Admin does not participate in selling or purchasing products.

3.2 Network Operator

Network Operators manage storefronts. Capabilities include attaching custom domains, curating vendor products, defining commission rules, issuing operator-funded coupons, running promotions, managing creators, and configuring storefront design. Operators pay subscription fees to access the platform.

Operators may also act as Vendor, Creator, or Customer.

3.3 Vendor

Vendors own product catalogs. Responsibilities include syncing product catalogs, defining pricing and inventory, approving operator relationships, defining minimum commissions, issuing vendor coupons, and receiving payouts.

Vendors may also act as Network Operator, Creator, or Customer.

3.4 Creator

Creators promote products using referral links. Capabilities include generating referral links, curating collections, creating creator-funded coupons, viewing earnings analytics (including click-through rate, conversion rate, and earnings-per-click), and accessing referral trees. Creators cannot access customer personal data.

Creators may also act as Vendor, Network Operator, or Customer.

3.5 Customer

Customers purchase products from storefronts. Customers can browse storefronts, purchase products, view order history, and track order status. Customer accounts work across all storefronts.

4. Multi-Role Account Model

Each user has a single global account. Users may hold multiple roles simultaneously: Operator, Vendor, Creator, Customer.

Users switch roles using a role selector in the interface. Role switching changes permissions and dashboard context. Authentication remains active during role switching.

4.1 Session and Context Scoping

Role context is session-scoped, not tab-scoped. A user may only be active in one role per session at a time. If a user opens a second session (e.g., another browser tab), both sessions share the same active role.

4.2 Conflict of Interest Rules

The platform must enforce conflict-of-interest guards when a user holds multiple roles:

- A user who is both Vendor and Operator may not approve their own products for listing. A separate Operator user must approve.
- A user who is both Creator and Vendor may not earn commission on their own products unless explicitly enabled by the Operator.
- Admin actions on entities owned by the Admin user in other roles must be audit-logged with elevated scrutiny.

NEW: Session scoping and conflict-of-interest rules added to prevent self-dealing and ambiguous multi-role states.

5. Storefront Model

Each storefront belongs to exactly one Network Operator. Storefront features include custom domains, branding and layout customization, product listings, collections, checkout, customer order history, and promotion links.

Operators may attach multiple domains to a storefront.

If an operator subscription becomes suspended, the storefront remains visible but checkout becomes disabled and a user-friendly notice is displayed.

6. Architecture

6.1 Technology Stack

Layer	Technology	Notes
Frontend	Next.js	Storefront + dashboards
Backend API	NestJS or Fastify	REST + versioned endpoints
Database	PostgreSQL	Primary data store
Queue / Cache	Redis	Caching, rate limiting, job queues
Message Broker	RabbitMQ or Kafka	Event-driven decoupling
Payments	Stripe Connect + Billing	Multi-party payments
Search	OpenSearch or Meilisearch	Required for product discovery
Storage	S3-compatible	Media and file storage
Monitoring	Prometheus + Grafana or Datadog	Metrics, alerts, dashboards
Feature Flags	LaunchDarkly or Unleash	Progressive rollout control

CHANGED: Search upgraded from optional to required. Message broker, monitoring stack, and feature flags added.

6.2 Monorepo Structure

The project uses a monorepo organized as follows:

- apps/ — api, storefront, dashboard, admin
- packages/ — contracts (API types/schemas), db (migrations, models), shared (utilities), sdk (client SDK)
- infra/ — deployment configs, environment configs, IaC templates

6.3 Non-Negotiable Platform Patterns

Multi-Tenancy

Tenant equals operator storefront. The `operator_id` is resolved from the request domain. Every database query on tenant-scoped data must include `operator_id` in its WHERE clause. Row-level security (RLS) in PostgreSQL is recommended as an additional safeguard.

Money Handling

All money flows are append-only ledger entries. Balances are derived from ledger entries, never stored or updated directly. Direct balance updates are strictly prohibited. All monetary values are stored as integers in the smallest currency unit (e.g., cents for USD). Rounding is applied once

at the final step of commission calculation, with fractional remainders resolving in favor of the vendor.

NEW: Integer currency storage, single-pass rounding, and remainder resolution rules defined.

Webhook Finality

Payments and subscriptions are finalized only after Stripe webhook confirmation. No order status may advance to confirmed until the corresponding Stripe event is received and verified.

Idempotency

All mutating endpoints that affect financial state (order creation, payment processing, ledger writes, refunds) must accept an Idempotency-Key header. The platform must store idempotency keys for a minimum of 24 hours. Duplicate requests within this window must return the original response without re-executing side effects.

NEW: Full idempotency requirements added for financial safety.

Event-Driven Architecture

The platform uses an event bus (message broker) to decouple core workflows. The following domain events must be published:

- order.created, order.paid, order.refunded, order.fulfilled
- commission.calculated, commission.cleared, commission.paid
- coupon.redeemed, coupon.reversed
- subscription.activated, subscription.suspended, subscription.cancelled
- catalog.synced, product.published, product.unpublished

Events are consumed asynchronously by dedicated workers for notification delivery, analytics aggregation, search index updates, and audit logging.

NEW: Event-driven architecture added to decouple processing and improve scalability.

7. API Standards

7.1 Versioning

All public API endpoints must be prefixed with a version identifier (e.g., /api/v1/). Breaking changes require a new major version. The platform must support at least two concurrent major versions. A deprecation notice must be issued at least 90 days before a version is retired. Internal APIs between services do not require versioning but must maintain backward-compatible contracts.

7.2 Error Response Format

All API errors must conform to the following unified schema:

- code: machine-readable error code (e.g., COMMISSION_EXCEEDS_MARGIN)
- message: human-readable description
- details: optional object with field-level errors or context
- traceId: unique request trace identifier for debugging

Error categories: validation (400), authentication (401), authorization (403), not_found (404), conflict (409), rate_limited (429), internal (500).

7.3 Rate Limiting

Rate limits are enforced per API key or session token. Limits are defined per role and endpoint category:

Role	Read Endpoints	Write Endpoints	Payment Endpoints
Admin	1000/min	200/min	50/min
Operator	500/min	100/min	30/min
Vendor	500/min	100/min	20/min
Creator	300/min	50/min	10/min
Customer	200/min	30/min	10/min

Burst allowance: 2x the per-minute rate for a 10-second window. Rate-limited responses return HTTP 429 with a Retry-After header.

7.4 Pagination

All list endpoints must support cursor-based pagination using after and limit parameters. Default page size is 25; maximum is 100. Responses include a hasMore boolean and a nextCursor value.

NEW: Entire Section 7 is new. Covers versioning, error format, rate limiting, and pagination.

8. Catalog and Product Management

Vendors are the source of truth for product catalogs. Operators can list products only after vendor approval.

Catalog sync must support scheduled synchronization (configurable interval, minimum 15 minutes), manual synchronization (on-demand trigger), inventory updates (real-time via webhook or polled), and product lifecycle updates (publish, unpublish, archive).

Orders are pushed back to vendor systems where integrations exist. Push failures are retried with exponential backoff (max 5 retries) and logged for manual reconciliation.

8.1 Search and Discovery

Product search is a required platform component. The search index must support full-text search across product titles, descriptions, and tags, faceted filtering by category, price range, vendor, and availability, and storefront-scoped results (search only returns products listed in the current storefront).

Search index sync lag must not exceed 30 seconds from product update to searchability. Index rebuilds must be supported without downtime.

NEW: Search elevated to required. Index sync SLA and faceting requirements defined.

9. Vendor-Operator Relationships

Relationships may be initiated by either party. Approval is required before an operator can list vendor products.

Approval may be catalog-wide (all current and future products) or product-specific (granular control per SKU). Vendors may revoke approval at any time; revocation triggers immediate unpublishing of affected products from the operator storefront.

10. Promote and Earn System

Creators generate permanent referral links. Referral links may point to products, collections, or storefronts. Referral links contain pseudonymous referral codes.

Referral trees allow users to view invited users and second-level referrals. Referral attribution respects user consent settings.

10.1 Attribution Window

The default attribution window is 30 days from first click. Attribution cookies are first-party and domain-scoped. Cross-device attribution is not supported in the initial release but must be architecturally planned for.

NEW: Attribution window and cookie scope defined.

11. Orders

Orders are scoped to storefront domains. Orders may contain multiple vendor items.

11.1 Order Visibility

- Customer: own orders only
- Vendor: own items only (no cross-vendor visibility)
- Operator: all orders within their storefront
- Creator: aggregated performance metrics only (no PII, no individual order details)

11.2 Order Snapshots

Orders include attribution metadata, commission snapshot, and coupon snapshot. Snapshots are captured at checkout time and are immutable. Refunds use the original snapshot values to ensure consistency regardless of subsequent commission or coupon rule changes.

11.3 Concurrency

Inventory decrements use optimistic locking with version numbers. If a concurrent update conflict is detected, the checkout request is retried once before returning an inventory conflict error. Ledger writes use serializable isolation within a database transaction to prevent double-crediting.

NEW: Concurrency control and locking strategy added for inventory and ledger operations.

12. Commission Management

Commissions can be defined globally (per storefront) or per product. Commission types supported: percentage of sale price, fixed amount per unit.

Participants: creator commission, operator margin, vendor earnings, platform transaction fee.

12.1 Commission Precedence

1. Product-level override (highest priority)
2. Global commission rule
3. Platform caps (lowest priority, always enforced)

12.2 Calculation Order

1. Start with product base price
2. Deduct platform fee (percentage of base price)
3. Deduct creator commission (percentage of base price)
4. Deduct operator margin (percentage of base price)
5. Remainder is vendor earnings

12.3 Rounding Rules

All monetary calculations use integer arithmetic in the smallest currency unit (e.g., cents). Rounding is applied once at the final calculation step. Fractional remainders (sub-cent values) are resolved in favor of the vendor. Each commission component is floored; the vendor receives the residual.

12.4 No-Loss Constraints

- Creator commission must not be negative.
- Operator margin must not be negative.
- Vendor earnings must meet the vendor-defined minimum.
- Platform fees must not be reduced below the platform-defined floor.

If any constraint is violated, the commission configuration must be rejected at save time with a descriptive validation error.

12.5 Worked Example

Product base price: \$100.00 (10000 cents). Platform fee: 5%. Creator commission: 10%. Operator margin: 15%.

Step	Calculation	Amount	Running Total
Base price	—	\$100.00	\$100.00
Platform fee	5% of \$100.00	-\$5.00	\$95.00
Creator commission	10% of \$100.00	-\$10.00	\$85.00
Operator margin	15% of \$100.00	-\$15.00	\$70.00

Vendor earnings	Remainder	\$70.00	\$0.00 remaining
-----------------	-----------	---------	------------------

If a 20% vendor coupon is applied (\$20.00 discount), the customer pays \$80.00 and the vendor absorbs the discount: vendor earnings become \$50.00. All other parties are unaffected.

NEW: Rounding rules, integer arithmetic requirement, and worked numerical example added.

13. Coupon Code Module

Coupons may be issued by Vendors, Operators, or Creators. Discount types: fixed amount or percentage.

Coupon discounts reduce only the issuer's earnings. If a coupon would reduce the issuer's earnings below zero, checkout must fail with a clear error message indicating insufficient issuer funds.

13.1 Coupon Scopes

- Storefront-wide (Operator-issued only)
- Vendor-scoped (applies to all products from a specific vendor)
- Product-scoped (applies to a single product)
- Collection-scoped (applies to all products in a collection)

13.2 Stacking Rules

Default rule: only one coupon per order. If coupon stacking is enabled by the Operator in the future, the following priority applies: product-scoped coupons first, then vendor-scoped, then storefront-wide. Combined discounts must still satisfy all no-loss constraints.

13.3 Expiry and Usage Limits

All coupons must include an expiry date. Optional usage limits: maximum total redemptions, maximum redemptions per customer. Expired or fully-redeemed coupons return a user-friendly error at checkout.

13.4 Refund Behavior

Refunds reverse coupon funding proportionally. The issuer's earnings are restored by the coupon discount amount.

13.5 Fraud Prevention

- Coupon creation is rate-limited: maximum 50 coupons per issuer per hour.
- Abnormal redemption patterns (e.g., same customer redeeming multiple codes rapidly) trigger automated alerts.
- Coupons with discount values exceeding configurable thresholds require secondary approval.

NEW: Stacking rules, fraud prevention, and usage limit requirements added.

14. Ledger and Earnings

The ledger is append-only. No existing entries may be modified or deleted. Corrections are made by appending reversal entries.

14.1 Ledger Entry States

- pending: payment received, awaiting clearance

- cleared: funds available for payout
- paid: payout executed to party
- reversed: refund or chargeback processed

14.2 Entry Types

Ledger entries exist for: vendor earnings, operator margin, creator commission, platform fee, coupon debits, refund reversals, and chargeback adjustments.

15. Payments and Payouts

Payments are processed using Stripe Connect (Standard or Express accounts for vendors and creators).

15.1 Payment Flow

1. Customer submits payment via Stripe Checkout or Payment Intents.
2. Platform fee is deducted via Stripe `application_fee_amount`.
3. Ledger entries are created for all parties (pending state).
4. Stripe webhook confirms payment (`payment_intent.succeeded`).
5. Ledger entries transition to cleared.
6. Transfers are executed to vendor and creator Stripe accounts.
7. Payout is executed per the party's configured payout schedule.

15.2 Refunds

Refunds reverse ledger entries and Stripe transfers proportionally. Partial refunds are supported at the line-item level. Refund amounts are calculated from the original order snapshot.

15.3 Webhook Handling

- All Stripe webhooks are verified using the webhook signing secret.
- Failed webhook processing is retried with exponential backoff: 1s, 5s, 30s, 5m, 30m (maximum 5 retries).
- Events that fail all retries are sent to a dead-letter queue for manual review.
- A daily reconciliation job compares Stripe records against the ledger and flags discrepancies.

NEW: Webhook retry policy, dead-letter queue, and reconciliation job added.

16. Subscription Model

Operators must subscribe to use the platform. Pricing model: tiered subscription plus per-transaction fee.

16.1 Subscription States

State	Storefront Visible	Checkout Enabled	Payouts Enabled
trial	Yes	Yes	No
active	Yes	Yes	Yes
past_due	Yes	Yes (grace)	Yes
grace_period	Yes	No	No
suspended	Yes	No	No
cancelled	No	No	No

enterprise	Yes	Yes	Yes
------------	-----	-----	-----

16.2 Enforcement Gates

Subscription status is checked during checkout, product publishing, referral link generation, and vendor onboarding. Grace period displays a banner to the Operator and disables checkout but keeps the storefront visible. Suspended operators cannot receive payouts; pending payouts are held until status is resolved.

17. Advanced Attribution Models

The platform supports multiple attribution models for creator commission allocation:

- Last click (default, launch model): 100% credit to the last referring creator
- First click: 100% credit to the first referring creator
- Linear: equal credit split across all contributing creators
- Time decay: exponentially weighted credit favoring recent interactions
- Weighted: operator-defined custom weight distribution

Attribution affects creator commission distribution only. Platform fees, operator margin, and vendor earnings are unaffected by the attribution model.

17.1 Implementation Phasing

- Phase 1: last-click only. All infrastructure built to support future models.
- Phase 2: first-click and linear added.
- Phase 3: time-decay and weighted added.

Model changes apply to new orders only. Historical orders retain their original attribution snapshot.

NEW: Phased rollout plan and forward/backward application rules added.

18. Promotions and Bundles

Operators may create product bundles containing items from multiple vendors. Bundles display a combined price that is less than or equal to the sum of individual prices.

Cart promotions include buy-X-get-Y, spend-X-get-discount, and tiered volume promotions. All promotion funding sources must respect no-loss rules; the party funding the promotion absorbs the discount from their earnings share.

19. Creator Growth Features

Creators have public pages displaying profile information, curated products, collections, and promotion links.

Optional features include leaderboards (opt-in, ranked by earnings or conversions) and performance-based commission boosts (automatically triggered when a creator exceeds defined thresholds).

19.1 Creator Analytics

Creator dashboards display the following KPIs:

- Click-through rate (CTR) per link
- Conversion rate (clicks to purchases)
- Earnings per click (EPC)
- Total earnings (pending, cleared, paid)
- Top-performing products and storefronts

Analytics data refreshes at minimum every 15 minutes. Historical data is retained for 24 months.

NEW: Specific KPIs, refresh rate, and retention period defined for creator analytics.

20. Vendor Growth Tools

Vendors can access analytics showing creator performance per product, conversion rates by storefront, and sales volume per creator.

Vendors may create campaigns including commission boosts (temporary increased commissions for specific products), targeted coupons (vendor-funded discounts distributed through creators), and featured placement bids (priority positioning in operator storefronts, where supported).

21. Operator Power Tools

Operators receive white-label analytics dashboards showing storefront revenue, order volume, top creators, and top products. Additional tools include vendor priority controls (ordering and featuring vendors), optional catalog sharing with other operators (requires mutual consent), and storefront theme customization (branding, layout, color schemes).

22. Enterprise Features

Enterprise capabilities include:

- Escrow settlement holds with configurable release triggers
- Dispute mediation workflows with escalation paths and SLA tracking
- Advanced REST APIs with dedicated rate limit tiers
- Webhook ecosystem: configurable outbound webhooks for order, payment, and catalog events
- Manual billing contracts with custom invoicing schedules
- Dedicated account management and priority support

23. Security and Infrastructure

23.1 Core Security Requirements

- TLS 1.2+ encryption for all data in transit
- AES-256 encryption for sensitive data at rest (PII, payment tokens)
- Role-based access control (RBAC) enforced at the API layer
- Tenant isolation enforced via query scoping and row-level security
- Rate limiting at the API gateway level (see Section 7.3)
- Comprehensive audit logs for all state-changing operations
- Containerized deployment (Docker + Kubernetes recommended)

23.2 Infrastructure Resilience

- Circuit breakers for all external service calls (Stripe, vendor catalog APIs, search engine). Fallback behaviors: cached data for reads, queued retries for writes.
- Health check endpoints: /health (shallow) and /health/ready (deep, including database and Redis connectivity).
- Uptime SLA target: 99.9% for payment and checkout paths.
- Latency thresholds: P95 under 500ms for API responses, P99 under 1500ms.
- Alerting: triggered on error rates exceeding 1%, latency exceeding thresholds, or queue depth exceeding configurable limits.

NEW: Circuit breakers, health checks, SLA targets, latency thresholds, and alerting rules added.

23.3 Disaster Recovery

- Database backups: automated daily full backups, continuous WAL archiving for point-in-time recovery.
- Backup retention: 30 days for daily backups, 7 days for WAL archives.
- RTO (Recovery Time Objective): 1 hour for full platform recovery.
- RPO (Recovery Point Objective): 5 minutes maximum data loss.
- Database replication: at least one synchronous read replica for failover.
- Disaster recovery runbook must be documented and tested quarterly.

NEW: Complete disaster recovery section added with RTO/RPO targets.

24. Database Management

24.1 Migration Strategy

Database schema changes are managed using a dedicated migration tool (e.g., Prisma Migrate or Flyway). All migrations must be backward-compatible: the previous application version must be able to run against the new schema. This enables zero-downtime deployments.

Migration requirements:

- Every migration must have a corresponding rollback script.
- Destructive changes (column drops, table drops) are executed in a two-phase process: Phase 1 deprecates (stop writing), Phase 2 removes (after verification period).
- All migrations are tested against a production-sized dataset in staging before production deployment.
- Migration execution time limits: individual migrations must complete within 5 minutes on production-sized data.

NEW: Entire database management section added with migration strategy and rollback procedures.

25. Caching Strategy

Redis is used for caching, session storage, and rate limit counters.

25.1 Cache Targets and TTLs

Data	Pattern	TTL	Invalidation Trigger
Product catalog	Cache-aside	10 minutes	Catalog sync event
Commission rules	Cache-aside	5 minutes	Rule update event
Session data	Write-through	24 hours	Logout or expiry
Storefront config	Cache-aside	15 minutes	Config update event
Rate limit counters	Direct write	1 minute	Window expiry
Search results	Cache-aside	2 minutes	Index update event

Cache invalidation is event-driven: domain events published to the message broker trigger cache eviction in subscribed services. Stale cache reads during invalidation windows are acceptable for non-financial data.

NEW: Complete caching strategy with specific targets, TTLs, and invalidation triggers.

26. Notifications and Messaging

The platform supports three notification channels: transactional email, push notifications (web), and in-app notifications.

26.1 Notification Types

- Order confirmation, shipping updates, delivery confirmation (Customer)
- New order received, low inventory alert (Vendor)
- Commission earned, payout processed, referral conversion (Creator)
- Subscription renewal, payment failed, new vendor application (Operator)
- Dispute filed, compliance alert (Admin)

26.2 Delivery and Preferences

All notification types can be individually enabled or disabled by the user in their preferences. Transactional emails related to payments and security (e.g., password reset, payout confirmation) cannot be disabled. Email delivery uses a dedicated transactional email service (e.g., SendGrid, Postmark) with delivery tracking and bounce handling.

NEW: Complete notification system architecture added.

27. File and Media Management

Product images and media files are stored in S3-compatible object storage and served via a CDN.

- Maximum upload size: 10 MB per image, 50 MB per file.
- Supported image formats: JPEG, PNG, WebP. Images are automatically converted to WebP for delivery.
- Image processing pipeline: resize to predefined breakpoints (thumbnail 150px, medium 600px, large 1200px) on upload.
- CDN cache TTL: 30 days for product images (cache-busted on update via content hash in URL).
- Media cleanup: images associated with deleted products are soft-deleted and permanently removed after 90 days.

NEW: Complete media management section added with upload limits, processing pipeline, and cleanup lifecycle.

28. Testing Strategy

The platform follows a testing pyramid approach.

28.1 Testing Levels

Level	Coverage Target	Scope
Unit tests	80% line coverage	Business logic, commission calc, coupon validation
Integration tests	Critical paths	API endpoints, database queries, Stripe integration
End-to-end tests	Core workflows	Checkout flow, refund flow, subscription lifecycle
Load tests	Pre-release	Checkout under 1000 concurrent users, search under load

28.2 Test Environment

A staging environment must mirror production configuration (including Stripe test mode, database schema, and Redis). Test data factories must produce realistic multi-tenant datasets. Stripe test clocks are used for subscription lifecycle testing.

NEW: Complete testing strategy added with coverage targets and environment requirements.

29. Internationalization

29.1 Currency

Each storefront operates in a single currency, configured by the Operator. The platform supports all Stripe-supported currencies. Cross-currency transactions (e.g., a vendor in EUR receiving payouts from a USD storefront) are handled by Stripe's automatic currency conversion. Commission calculations are always performed in the storefront's currency.

29.2 Language and Localization

The platform UI supports multiple languages via an i18n framework. Product content (titles, descriptions) is stored in the vendor's language; translation is the vendor's or operator's responsibility. Date, time, and number formatting respect the user's locale settings. All timestamps are stored in UTC and displayed in the user's local timezone.

29.3 Tax Considerations

Tax calculation is delegated to Stripe Tax or a third-party tax service. The platform does not implement its own tax engine. Tax amounts are displayed at checkout and recorded as a separate line item in orders.

NEW: Complete internationalization section added covering currency, language, and tax.

30. Mobile Strategy

The initial release targets responsive web (PWA-ready). Native mobile apps are planned for Phase 4. The storefront must be fully functional on mobile viewports (minimum 320px width). API responses support partial field selection via a fields query parameter to reduce payload size for mobile clients. Image delivery uses responsive srcset attributes to serve appropriately sized images per viewport.

NEW: Mobile strategy section added with responsive requirements and API optimization.

31. Audit and Observability

The platform must log all financial transactions, commission configuration changes, coupon creation and redemption, subscription lifecycle events, admin actions, role changes, and data access events for sensitive resources.

Audit logs are append-only and stored separately from application data. Logs must include actor identity, action type, resource identifier, timestamp, and before/after state for mutations. Logs must be retained for a minimum of 7 years for financial records and 3 years for non-financial records. Logs must be exportable in structured format (JSON or CSV) for regulators.

31.1 Observability Stack

Application metrics (request rate, error rate, latency percentiles) are collected via Prometheus-compatible instrumentation. Dashboards are provisioned for platform health, storefront performance, payment success rates, and queue depth. Distributed tracing (OpenTelemetry) is required for all cross-service requests. Structured application logging (JSON format) with correlation IDs linking related events across services.

NEW: Observability stack requirements, retention periods, and tracing added.

Appendix A — GDPR and Privacy Compliance

Operators act as Data Controllers. The platform acts as Data Processor. Payment providers (Stripe) are Sub-processors.

Cookie Consent

Cookie consent must support three categories: necessary (always active), analytics (opt-in), and marketing (opt-in). Consent must be domain-scoped (per storefront), require explicit opt-in (no pre-checked boxes), and be revocable at any time through the storefront UI.

Data Subject Rights

The platform must support the following rights with a maximum response time of 30 days per GDPR Article 12:

- Access: export of all personal data in machine-readable format (JSON)
- Deletion: anonymization of personal data; transactional records required by law are retained with PII removed
- Correction: self-service profile editing plus admin-assisted corrections
- Portability: data export in standard format
- Objection: opt-out of analytics and marketing processing

NEW: 30-day SLA and detailed implementation approach per right added.

Data Retention

Orders are retained per statutory requirements (typically 7 years for financial records). Referral logs are minimized: click-level data is aggregated after 90 days, retaining only summary statistics. Inactive user accounts are anonymized after 24 months of inactivity (with 30-day advance notice).

Referral Tree and Deletion

When a user requests deletion, their node in the referral tree is anonymized (replaced with a placeholder ID). Child referral relationships are preserved with the anonymized parent. Commission history is retained with PII removed.

NEW: Referral tree deletion handling specified.

Security

All personal data is encrypted at rest (AES-256) and in transit (TLS 1.2+). Access to PII is restricted by RBAC and logged. Breach notification procedures must comply with the 72-hour GDPR notification requirement. A Data Protection Impact Assessment (DPIA) must be completed before launch.

Appendix B — Glossary

Term	Definition
Storefront	A branded, domain-scoped online store operated by a Network Operator
Operator	A user who owns and manages one or more storefronts
Vendor	A user who owns product catalogs and fulfills orders
Creator	A user who promotes products via referral links and earns commissions
Ledger	Append-only financial record of all earnings, deductions, and payouts
Attribution	The process of assigning credit (and commission) to creators for a sale
Snapshot	Immutable copy of commission/coupon rules captured at order time
Idempotency Key	Client-provided token ensuring a request is processed at most once
RTO	Recovery Time Objective: maximum acceptable downtime after a failure
RPO	Recovery Point Objective: maximum acceptable data loss after a failure
DLQ	Dead Letter Queue: storage for events/messages that failed all processing retries
WAL	Write-Ahead Log: PostgreSQL mechanism enabling point-in-time recovery

NEW: Glossary appendix added for team alignment on terminology.